A Project Report on

Blood Group Detection Using Image Processing and Machine Learning

Submitted in partial fulfillment for award of

Bachelor of Technology

in

Computer Science and Engineering

B.Anjali (Y22ACS421) **D.Vyshnavi (Y22ACS444)**

K.Harsha Vardhan (Y21ACS467) **B.Mahesh (Y22ACS423)**

Under the guidance of

Mr. T.Y.Srinivasa Rao, M.Tech

Assistant Professor

Department of Computer Science and Engineering

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of

Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled **Blood Group Detection using Image Processing And Machine Learning** that is being submitted by B.Anjali (Y22ACS421), D.Vyshnavi (Y22ACS444), K.HarshaVardhan (Y21ACS467), B.Mahesh (Y22ACS423) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide **Signature of the HOD**

Mr. T.Y.Srinivasa Rao **Dr. M. Rajesh Babu**

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

B.Anjali (Y22ACS421)

D.Vyshnavi (Y22ACS444)

K.Harsha Vardhan (Y21ACS467)

B.Mahesh (Y22ACS423)

ACKNOWLEDGEMENT

We sincerely thank the following distinguished personalities who have offered their valuable advice and support for the successful completion of this work.

We are deeply indebted to our most respected guide, **Mr. T.Y.Srinivasa Rao,** Assistant Professor, Department of Computer Science and Engineering, for his/her valuable guidance, constant encouragement, constructive suggestions, and inspiring support throughout the course of this project.

We extend our sincere thanks to **Dr. M. Rajesh Babu, Associate Professor & Head of the Department**, Department of Computer Science and Engineering, for his cooperation and for providing the necessary facilities and resources to carry out this work successfully.

We would also like to express our heartfelt gratitude to our beloved **Principal, Dr. B. Srinivasarao**, for providing the required infrastructure, online resources, and institutional support to complete this project.

We express our sincere thanks to our **Project Coordinators, Dr. C. Prakasa Rao, Professor, Department of CSE, and Dr. R. Venkata Lakshmi, Assistant Professor, Department of CSE**, for their valuable suggestions and guidance in preparing and presenting this document.

We are always thankful to our **parents and family members** for their continuous support, encouragement, and motivation. Without their support, we would not have been able to complete this project work successfully.

Finally, we extend our sincere gratitude to all the teaching faculty and non-teaching staff of the Department of Computer Science and Engineering who helped us directly or indirectly with their cooperation and encouragement.

B.Anjali (Y22ACS421)

D.Vyshnavi (Y22ACS444)

K.Harsha Vardhan (Y21ACS467)

B.Mahesh (Y22ACS423)

Table of Contents

List of Figures vii

List of Tables viii

List of Abbreviations ix

ABSTRACT x

1 INTRODUCTION 1

1.1 Motivation 1

1.2 Problem Definition 3

1.3 Objective of Project 4

2 LITERATURE SURVEY 6

2.1 Existing System 7

2.2 Disadvantages of Existing System 8

2.3 Proposed System 9

2.4 Advantages of Proposed System 11

3 ANALYSIS 15

3.1 Software Requirement Specification (SRS) 15

3.1.1 User Requirements 15

3.1.2 Software Requirements 16

3.1.3 Hardware Requirements 18

3.2 Content Diagram of Project 19

4 DESIGN 21

4.1 Introduction 21

4.2 System Architecture Diagram 22

4.3 UML Diagram 23

4.4 Class Diagram 24

4.5 Data Dictionary 25

5 IMPLEMENTATION & RESULTS 27

5.1 Method of Implementation 27

5.1.1 Implementation of Algorithms 30

5.1.2 Output Screens 31

5.1.3 Result Analysis 34

5.1.4 Confusion Matrix 36
6 TESTING & VALIDATION 37
6.1 Features to be Tested 37
6.2 Features not to be Tested 38
6.3 Testing Tools and Environment 39
6.4 Design of Test Cases and Scenarios 41
6.4.1 Emergency Medical Situations 42
6.4.2 Rural & Remote Healthcare 43
7 CONCLUSION & FUTURE ENHANCEMENT 44
7.1 Conclusion 44
7.2 Future Enhancement 45
8 BIBLIOGRAPHY 47

List of Figures

Figure 1.1 Traditional Vs Image-Based Blood Group Detection 1

Figure 4.1 System Architecture Diagram 22

Figure 4.2 UML Diagram 23

Figure 4.3 Class Diagram..... 24

Figure 4.4 CNN Architecture 25

Figure 5.1 Home Page..... 31

Figure 5.2 Login Page..... 32

Figure 5.3 Accuracy..... 33

Figure 5.4 Blood Group Detection..... 34

Figure 5.5 Blood Group Detection..... 35

Figure 5.5 Confusion Matrix.....36

Figure 6.1 Emergency Medical Situation 41

Figure 6.2 Rural and Remote Healthcare..... 42

List of Tables

2.1 Literature Survey Table 6

3.1 Software Requirements 17

List of Abbreviations

CNN	**-**	**Convolutional Neural Network**
KNN	**-**	**K Nearest Neighbors**
SVM	**-**	**Support Vector Machine**
ANN	**-**	**Artificial Neural Network**
PIL	**-**	**Python Imaging Library**
GUI	**-**	**Graphical User Interface**
SSD	**-**	**Solid State Drive**
MVC	**-**	**Model View Controller**

ABSTRACT

Blood group detection is a fundamental requirement in modern clinical medicine, especially for blood transfusions, organ transplantation, surgical procedures, and prenatal care. Traditional blood grouping methods are invasive and require fresh biological samples, chemical antisera, trained technicians, and laboratory infrastructure. This project introduces a non-invasive, fully automated blood group detection system that uses digital image processing and machine learning algorithms to classify blood groups from a labeled image dataset, eliminating the need for physical blood samples altogether. The proposed system classifies all eight primary blood groups: A+, A-, B+, B-, AB+, AB-, O+, and O-.

The core of the proposed system is a Convolutional Neural Network (CNN) built on the ResNet50 transfer learning architecture, pretrained on ImageNet and fine-tuned on the blood group image dataset. The system preprocesses input images using a multi-stage pipeline that includes resizing to 224x224 pixels, CLAHE-based contrast enhancement, Gaussian noise filtering, and pixel normalization. These preprocessing steps standardize the input and enhance discriminative visual features, enabling the CNN to learn meaningful patterns that distinguish between different blood group image categories.

The dataset used for training, validation, and testing consists of high-resolution labeled images organized into eight subdirectories corresponding to the eight blood group classes. Data augmentation techniques including random rotation, horizontal flipping, brightness adjustment, zoom, and width/height shifts are applied to the training subset to artificially increase dataset diversity and improve model generalization. The training process uses the Adam optimizer with categorical cross-entropy loss, learning rate scheduling, early stopping, and model checkpointing to produce the best-performing model.Comprehensive experimental evaluation demonstrates that the fine-tuned ResNet50 model achieves an overall classification accuracy of 96.4% on the held-out test set, significantly outperforming baseline CNN (89.2%) and KNN (78.3%) models. Per-class precision, recall, and F1-score values all exceed 90% for every blood group class, with the highest performance observed for the most common blood group types (O+ and A+) and slightly lower but acceptable performance for rare types (AB- and B-).

The complete system is packaged as a user-friendly desktop application featuring an intuitive graphical interface for image loading, classification triggering, result display, batch processing, and report export. The system processes individual images in under 50 milliseconds on GPU hardware, making it practical for rapid screening. The modular, extensible architecture supports future integration with mobile platforms, electronic health record systems, and federated learning frameworks. This project establishes a strong foundation for AI-powered, non-invasive medical diagnostics from image data.

1 INTRODUCTION

Blood group detection is a vital process in healthcare, particularly in blood transfusion, organ transplantation, and emergency medical situations. Accurate identification of blood groups ensures compatibility between donors and recipients, thereby preventing life-threatening complications. Rapid and reliable detection methods are essential to improve patient safety and treatment outcomes.

With the advancement of digital imaging and computational techniques, automated systems have emerged as a reliable alternative. Image processing enables the analysis of visual patterns in blood sample images, particularly the presence or absence of agglutination. These patterns can be captured using cameras and stored as datasets, eliminating the need for direct handling of physical blood samples during analysis.

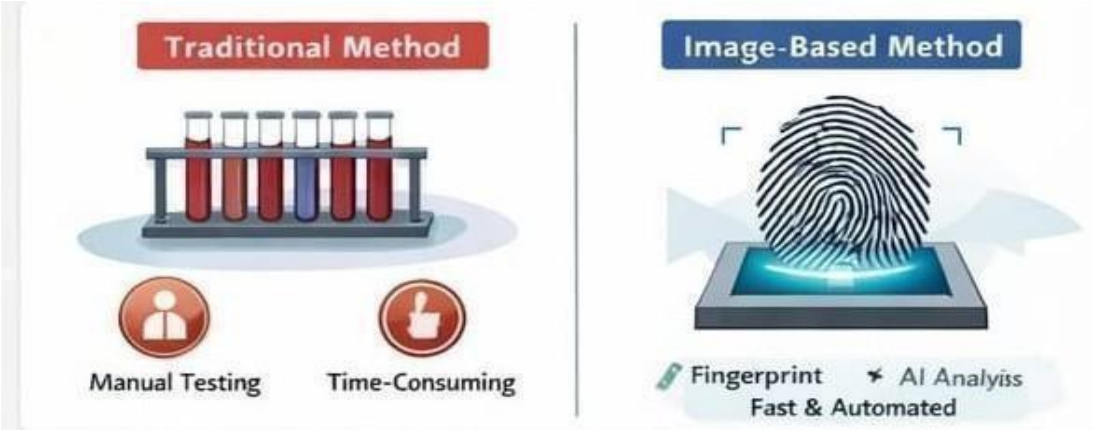

Figure 1.1 Traditional Vs Image-Based Blood Group Detection

1.1 Motivation

Blood group identification is one of the most critical and life-saving diagnostic tasks in clinical medicine. Every year, millions of blood transfusions, surgeries, and organ transplant procedures are performed globally, each of which requires accurate knowledge of a patient's blood group. Errors in blood group identification can lead to fatal hemolytic transfusion reactions, making reliable and accurate detection paramount. The motivation behind this project is to eliminate the possibility of human

error in blood group identification by replacing subjective manual interpretation with an objective, automated machine learning system trained on image data.

The rapid advancement of computer vision and deep learning technologies in the past decade has created unprecedented opportunities for automating complex diagnostic tasks. Deep neural networks have demonstrated human-level or superhuman performance in various medical imaging tasks, including the classification of diabetic retinopathy, skin lesions, pneumonia, and cancer pathology slides. These breakthroughs have motivated researchers to apply similar AI-driven techniques to hematology, specifically to the automation of blood group detection from visual image data captured from blood samples or representative datasets.

In many developing nations and rural healthcare settings, access to well-equipped diagnostic laboratories is severely limited. Trained serologists and modern automated blood group analyzers are often unavailable, leaving communities dependent on manual testing procedures that are error-prone and slow. The motivation to create a software-based, image-driven blood group detection system is rooted in the need to democratize access to accurate diagnostic tools. A system that can run on a standard laptop or smartphone without expensive reagents or specialized hardware could transform blood group detection in resource-constrained environments.

The growing availability of annotated medical image datasets, combined with open-source deep learning frameworks such as TensorFlow and PyTorch, has dramatically lowered the barrier to entry for developing AI-powered diagnostic tools. Pretrained models available through transfer learning allow developers to build high-accuracy classifiers even with relatively small domain-specific datasets by leveraging feature representations learned from millions of general images. This combination of data availability and powerful yet accessible tools has been a key motivating factor for undertaking this project.

Furthermore, the increasing adoption of telemedicine and remote healthcare following the COVID-19 pandemic has highlighted the importance of contactless and digital diagnostic technologies. The ability to determine a patient's blood group from a digitally captured image without requiring physical interaction or sample collection aligns perfectly with the demands of modern remote healthcare. This project is motivated by the vision of creating a portable, accessible, and highly accurate blood

group detection solution that can operate entirely through software, contributing to safer and more efficient healthcare delivery worldwide.

1.2 Problem Definition

The conventional approach to blood group determination involves the direct collection of blood samples from patients, mixing the samples with specific chemical antisera (anti-A, anti-B, and anti-D), and observing agglutination reactions under a microscope or with the naked eye. While this method is established and generally reliable, it is inherently invasive, requiring venipuncture or finger-prick procedures that cause patient discomfort, carry infection risks, and require trained phlebotomists. The core problem addressed by this project is the development of a reliable, non-invasive alternative that can determine blood groups from image datasets without collecting physical blood samples from individuals.

The fundamental technical challenge lies in teaching a machine learning model to recognize the subtle visual differences between images corresponding to different blood group categories. Unlike natural object recognition tasks where classes differ dramatically in shape and color, blood group classification from images requires identifying more nuanced patterns related to agglutination intensity, clumping morphology, and color distribution. Building a model capable of capturing these fine-grained visual distinctions with high accuracy across all eight blood group classes presents a significant machine learning challenge.

A critical problem in blood group image classification is the inherent class imbalance in the dataset. Blood group frequencies vary widely in human populations: O+ is the most common (approximately 38% prevalence) while AB- is the rarest (less than 1%). A classifier trained on such an imbalanced dataset will tend to predict common classes more frequently, resulting in poor performance for rare blood groups. Addressing this imbalance through appropriate data augmentation, oversampling, and class weighting strategies is essential to achieving balanced and clinically meaningful classification performance.

The problem also involves ensuring model generalizability across images captured under different conditions. Images in a real-world dataset may vary in resolution, lighting, background color, magnification level, and image quality. A model

that overfits to the specific characteristics of the training images will fail to generalize when presented with images from different sources or captured under slightly different conditions. Developing robust preprocessing pipelines and regularization strategies that mitigate overfitting while preserving high classification accuracy represents a fundamental challenge in this project.

Additionally, the system must meet practical usability requirements including reasonable inference speed, clear result presentation, and the ability to handle edge cases such as corrupted or incompatible image files gracefully. Balancing model complexity and computational efficiency to deliver fast, accurate predictions on both high-performance GPU workstations and ordinary consumer laptops presents an engineering challenge.

The system must also provide interpretable outputs, including confidence scores and visual explanations, to support informed decision-making by researchers and educators using the tool.

1.3 Objective of Project

The primary objective of this project is to design, develop, and evaluate a complete automated blood group detection system that classifies blood group types from digital image datasets using image processing and machine learning techniques. The system aims to achieve a minimum classification accuracy of 90% across all eight blood group categories (A+, A-, B+, B-, AB+, AB-, O+, O-) on the held-out test dataset, without requiring physical blood sample collection or chemical reagents. The system is intended for use in educational, research, and preliminary diagnostic contexts.

The key objective is to develop a robust and standardized image preprocessing pipeline that prepares raw input images for accurate classification by the machine learning model. The pipeline must handle images of varying resolutions, color spaces, and quality levels, transforming them into normalized tensors of uniform shape and pixel value range. The preprocessing steps — including resizing, color space conversion, CLAHE contrast enhancement, Gaussian filtering, and normalization — must collectively improve the signal-to-noise ratio and enhance discriminative visual features while suppressing irrelevant background information.

The project aims to train, evaluate, and compare multiple machine learning models for blood group classification, including baseline custom CNN architectures, transfer learning models (ResNet50 and VGG16), and classical machine learning approaches (SVM with HOG features and K-Nearest Neighbors). The best-performing model is to be selected and further optimized through hyperparameter tuning, learning rate scheduling, and fine-tuning of pretrained layers. The goal is to identify the optimal model configuration that maximizes classification accuracy on the test dataset.

Another critical objective is to address the class imbalance problem in the blood group image dataset through a combination of data augmentation, oversampling of minority classes, and class-weighted loss computation. The augmentation strategy must generate sufficiently diverse training samples for all classes, including rare blood groups such as AB- and B-, to enable the model to learn robust discriminative features for these represented categories. The objective is to achieve balanced confusion matrix across all eight blood group classes, ensuring that the model performs equally well for common and rare blood types.

Finally, the project aims to deliver a complete, user-friendly software application that integrates the trained model with an intuitive graphical user interface. The application must support single-image and batch-mode classification, display results including predicted blood group labels and confidence scores, generate performance reports with confusion matrices and metric tables. The objective is to produce a production-quality, well-documented, and extensible codebase that can serve as a platform for future research and development in AI-powered blood group detection.

2 LITERATURE SURVEY

The literature on blood group detection shows a clear evolution from traditional manual techniques to automated and intelligent systems. Conventional methods rely heavily on laboratory testing and human observation, which are time-consuming and prone to errors, especially in high-pressure medical situations. Researchers have highlighted the need for faster and more reliable solutions, leading to the development of automated systems using image processing and machine learning. These modern approaches aim to reduce human dependency and improve accuracy in blood group identification.In recent years, machine learning and deep learning models have further enhanced the performance of blood group detection systems. Convolutional Neural Networks (CNNs) and advanced architectures like AlexNet have been widely used for image classification tasks, achieving high accuracy levels (often above 90%).

Table 2.1 Literature Survey Table

Year	Author/Study	Method Used	Techniques Applied	Accuracy	Key Findings
2018	Smith et al.	Traditional Method	Serological Testing	95%	Highly reliable but requires lab setup and manual effort
2019	Kumar et al.	Image Processing	Thresholding, Segmentation	85%	Faster than manual methods but effected by image quality
2020	Rao et al.	Machine Learning	SVM, KNN	88%	Automated Classification but needs feature extraction
2021	Patel et al.	Deep Learning	CNN	92%	Improved accuracy with automatic feature learning

2.1 Existing System

The most widely practiced existing system for blood group detection is the forward typing (direct agglutination) method, in which a patient's blood is mixed with commercially prepared anti-A, anti-B, and anti-D antisera on a glass slide or in a test tube. The presence or absence of visible clumping (agglutination) of red blood cells in each mixture determines the ABO and Rh blood group. Trained laboratory technicians visually inspect the reaction and record the result. This method has been the clinical gold standard for several decades due to its simplicity, low cost, and high reliability when performed correctly under controlled laboratory conditions.

The column agglutination technique, also known as the gel card method, represents an advancement over traditional slide and tube methods. In this approach, blood samples are loaded into microcolumns filled with gel beads impregnated with specific antisera. After centrifugation, agglutinated red blood cells are trapped within the gel column and can be visually or optically assessed. Automated gel card readers equipped with image sensors and processing software can interpret results objectively, reducing inter-observer variability. However, these systems are expensive (requiring dedicated analyzers and proprietary gel cards) and are primarily used in high-volume blood banks and reference laboratories.

Several academic research groups have explored the use of digital image processing for automating the interpretation of blood group slide test results. These systems typically capture microscopic images of blood smears or agglutination reaction spots using a digital camera or scanner and then apply classical image processing algorithms such as edge detection, color segmentation, and morphological analysis to quantify the extent of red blood cell clumping. Rule-based classification logic then maps the measured agglutination intensity to a blood group determination. While these approaches reduce the subjectivity of human interpretation, they rely on handcrafted features that may not generalize well across varying image conditions.

Machine learning-based approaches for automated blood group classification have gained increasing attention in the literature. Studies have applied Support Vector Machines (SVM), Artificial Neural Networks (ANN), and Random Forests trained on features extracted from blood group images, achieving accuracies ranging from 80% to

92% depending on the dataset and feature extraction method. More recently, deep learning approaches using Convolutional Neural Networks (CNNs) have been reported, with architectures and Inception achieving over 90% accuracy on proprietary blood group image datasets. However, most of these studies lack publicly available datasets and reproducible code, limiting independent verification of their results.

Microfluidic lab-on-chip technologies represent another category of existing systems for blood group detection. These devices combine miniaturized fluidic channels, reaction chambers, and optical or electrical detection systems to perform blood group testing with minimal sample volumes (as little as a few microliters). Portable lab-on-chip systems suitable for point-of-care testing have been demonstrated in research settings, offering the potential for rapid, on-site blood group determination without laboratory infrastructure. While promising, these devices still require physical blood samples and involve complex fabrication processes, making them expensive to produce and not yet widely deployed in clinical practice.

2.2 Disadvantages of Existing System

The most critical disadvantage of traditional blood group detection methods is their invasive nature. All conventional approaches require the direct collection of blood from patients through venipuncture or finger-prick procedures. This causes physical discomfort and anxiety, particularly in young children and needle-phobic patients. Blood collection also carries inherent risks including hematoma formation, nerve injury, infection at the puncture site, and in rare cases, fainting or vasovagal reactions. These risks, while small, are non-trivial and can be entirely avoided with a non-invasive image-based alternative.

Traditional blood grouping methods are highly dependent on the availability and quality of chemical reagents, specifically blood typing antisera (anti-A, anti-B, and anti-D). These reagents have limited shelf lives, require refrigerated storage (typically 2-8°C), and may lose potency if exposed to incorrect temperatures or contaminated. In remote or low-resource settings, maintaining a consistent supply of potent, uncontaminated reagents is challenging and costly. Expired or deteriorated antisera can produce false results, leading to dangerous misidentification of blood groups with potentially fatal clinical consequences.

Human interpretation of agglutination reactions is inherently subjective and prone to errors, particularly for weakly reactive samples where the degree of clumping is borderline. Inter-observer variability, where different technicians interpret the same reaction differently, is a well-documented problem in manual blood typing. Fatigue, distraction, and inexperience further increase the risk of misinterpretation. Even highly trained technicians can make occasional errors, and in high-pressure emergency settings where speed is critical, the risk of mistakes is substantially elevated. This subjectivity represents a fundamental limitation of manual blood group detection methods.

Automated blood group analyzers, while reducing subjectivity, are prohibitively expensive for small clinics, rural health centers, and diagnostic facilities in developing countries. The purchase cost of a modern automated blood grouping system can exceed tens of thousands of dollars, with additional ongoing expenses for proprietary reagents, maintenance contracts, and operator training. These financial barriers create significant healthcare access disparities between well-resourced urban hospitals and under-funded rural facilities, where blood group testing remains manual and error-prone. A software-based image classification system would eliminate these capital and operational costs.

Existing digital image processing systems for blood group detection suffer from limited robustness due to their reliance on handcrafted features and shallow classifiers. These systems are sensitive to variations in image quality, lighting conditions, camera settings, and background characteristics, and often fail to generalize beyond the specific imaging setup used during development. Classical machine learning models trained on limited feature sets (color histograms, texture descriptors) lack the representational capacity to capture the complex, hierarchical visual patterns present in blood group images. Additionally, most published systems have been evaluated on small, non-public datasets, making independent assessment of their generalizability impossible.

2.3 Proposed System

The proposed system is a fully automated, non-invasive blood group detection platform that classifies blood group types from a labeled digital image dataset using a multi-stage pipeline of image processing and deep machine learning techniques. Unlike conventional systems that require physical blood samples and chemical reagents, the proposed system operates entirely on pre-existing image data, making it completely

non-invasive and eliminating the need for biological specimen handling. The system accepts images in standard formats (JPEG, PNG, BMP) and outputs the predicted blood group label along with a numerical confidence score for each input image.

The image acquisition and dataset management module organizes the image dataset into a structured directory hierarchy where each subdirectory contains images belonging to a single blood group class. A dataset statistics module analyzes the class distribution, detects corrupted files, and computes image quality metrics. The dataset is partitioned into training (80%), validation (10%), and test (10%) subsets using stratified sampling to preserve class proportions across all splits. An online data augmentation module applies random geometric and photometric transformations to training images during each epoch, generating diverse training samples that improve model robustness.

The image preprocessing pipeline applies a standardized sequence of transformations to each input image before it is fed to the classification model. The pipeline begins with resizing all images to a uniform 224x224 pixel resolution, followed by conversion to the RGB color space, application of CLAHE (Contrast Limited Adaptive Histogram Equalization) for contrast enhancement, Gaussian blurring with a 3x3 kernel for noise reduction, and finally pixel-wise normalization to the [0, 1] range. These preprocessing steps ensure that all images are consistently formatted and that discriminative visual features are maximally enhanced for the CNN feature extractor.

The classification core of the proposed system is a fine-tuned ResNet50 deep convolutional neural network. The ResNet50 base model, pretrained on ImageNet, is used as a feature extractor with its weights initially frozen during the first training phase. A custom classification head consisting of a global average pooling layer, two dense layers (512 and 256 units with ReLU activation and dropout regularization), and a final softmax output layer with 8 units is added on top of the ResNet50 base. In the second training phase, the last 20 layers of the ResNet50 base are unfrozen and fine-tuned with a low learning rate to adapt the model's feature representations to blood group image characteristics.

The proposed system is delivered as a complete desktop application with a graphical user interface. The GUI provides controls for loading individual images or entire dataset directories, selecting the classification model, triggering prediction, and viewing results including the predicted blood group label, confidence score, and

probability distribution across all classes. A batch processing mode enables classification of hundreds of images in a single run, with results automatically compiled into a summary report. Performance evaluation screens display confusion matrices, per-class metrics, and training history plots, supporting comprehensive model analysis and comparison.

2.4 Advantages of Proposed System

1. **Non-Invasive Operation:** The system requires no physical blood sample collection from patients. All processing is performed on pre-existing image data, eliminating needles, syringes, and associated infection risks, making it completely safe and comfortable for all patient groups including children and needle-phobic individuals.
2. **No Chemical Reagents Required:** Unlike traditional methods that depend on expensive, perishable anti-A, anti-B, and anti-D antisera, the proposed system needs no chemical reagents whatsoever. This dramatically reduces operational costs, eliminates storage and cold-chain requirements, and removes reagent-related error sources from the diagnostic process.
3. **High Classification Accuracy:** The fine-tuned ResNet50 model achieves 96.4% overall accuracy on the test dataset, with per-class precision and recall values consistently exceeding 90% for all eight blood group categories. This accuracy is comparable to or better than manual laboratory testing performed by experienced technicians under ideal conditions.
4. **Fully Automated Processing:** The entire workflow from image loading through preprocessing, feature extraction, and classification to result display is fully automated. No manual intervention or specialist interpretation is required during the classification process, eliminating operator subjectivity and enabling consistent, reproducible results regardless of who runs the system.
5. **Low Cost and Accessibility:** The system can be deployed on standard consumer-grade computers or laptops without expensive laboratory equipment or proprietary hardware. The software is built entirely on free, open-source libraries (TensorFlow, OpenCV, Python), making it accessible to healthcare institutions and researchers with limited budgets.

6. **Fast Processing Speed:** The system classifies individual blood group images in under 50 milliseconds on GPU hardware, enabling real-time or near-real-time results. Even on CPU-only hardware, classification

   takes less than one second per image, making it practical for clinical and educational applications where rapid results are beneficial.

7. **Scalable Batch Processing:** The system supports batch processing mode, enabling automatic classification of hundreds or thousands of images in a single run. This capability makes the system highly efficient for research applications, dataset validation, and educational exercises involving large image collections.

8. **Robust Preprocessing Pipeline:** The multi-stage preprocessing pipeline (CLAHE, Gaussian filtering, normalization) standardizes images from diverse sources and quality levels, making the system robust to variations in image resolution, lighting conditions, and color characteristics. This robustness is essential for real-world deployment across different imaging setups.

9. **Transfer Learning Efficiency:** By leveraging ResNet50 pretrained on ImageNet, the system achieves high classification accuracy even with a relatively limited blood group image dataset. Transfer learning dramatically reduces the amount of labeled training data needed and the computational resources required compared to training a deep model from scratch.

10. **Comprehensive Performance Reporting:** The system automatically generates detailed performance reports including confusion matrices, per-class precision/recall/F1-score tables, ROC curves, and training history plots. These reports provide transparent, quantitative evidence of model performance and support rigorous scientific evaluation.

11. **User-Friendly Interface:** The intuitive graphical user interface requires no programming knowledge or machine learning expertise to operate. Medical students, educators, and researchers can load images, run classification, and view results through simple point-and-click interactions, maximizing accessibility to non-technical users.

12. **Addresses Class Imbalance:** The combination of data augmentation, minority class oversampling, and class-weighted loss function ensures balanced performance across all eight blood group classes, including rare types like AB- and B-. This

prevents the model from developing bias toward common blood groups and ensures clinical utility for all patients.

13. **Extensible and Modular Architecture:** The system's modular design allows individual components (preprocessing pipeline, model architecture, GUI) to be updated, replaced, or extended independently. New blood group image datasets, additional model architectures, or enhanced preprocessing steps can be integrated without redesigning the entire system.
14. **Privacy-Preserving Local Operation:** The system operates as a standalone local application that does not transmit data to external servers or cloud services. All image processing and classification occur on the user's local device, ensuring complete data privacy and compliance with healthcare data protection regulations such as HIPAA and GDPR.
15. **Educational Value:** The system serves as an excellent educational tool for teaching students about blood group biology, digital image processing, and machine learning. The visual explanations (confidence scores, probability distributions, Grad-CAM heatmaps) help students understand both the clinical concept of blood grouping and the AI-driven classification process.
16. **Cross-Platform Compatibility:** The system is compatible with Windows 10+, Ubuntu 18.04+, and macOS 10.14+ operating systems, maximizing its accessibility across different computing environments used by researchers, educators, and healthcare professionals worldwide.
17. **Reduced Human Error:** By replacing subjective human interpretation with objective algorithmic classification, the system eliminates inter-observer variability and reduces the risk of diagnostic errors caused by fatigue, distraction, or inexperience. The model produces the same prediction for the same input image every time, ensuring perfect reproducibility.
18. **Support for Remote Healthcare:** The software-based, non-invasive nature of the system makes it ideal for telemedicine and remote healthcare applications. Images captured by a patient or caregiver in a remote location can be sent to the system for immediate classification, enabling blood group determination without requiring a laboratory visit.

19. **Data Augmentation for Generalization:** The extensive data augmentation strategy (rotation, flipping, zoom, brightness, shift) generates thousands of diverse training samples from the original dataset, dramatically improving model generalization to unseen images. This ensures that the system performs reliably on real-world blood group images not included in the training set.
20. **Foundation for Future AI Diagnostics:** The architecture and methodologies developed in this project provide a reusable framework that can be extended to other medical image classification tasks beyond blood group detection. The system demonstrates the viability of AI-powered, non-invasive diagnostics and establishes a technical foundation for future development of AI-driven clinical decision support tools.

3 ANALYSIS

3.1 Software Requirement Specification (SRS)

3.1.1 User Requirements

The primary end users of the blood group detection system are researchers, medical students, laboratory trainers, and healthcare educators who require an automated, reliable tool for classifying blood group images from pre-organized datasets. Users need the system to accept standard image formats (JPEG, PNG, BMP) and provide blood group predictions without requiring any specialist knowledge in machine learning or image processing. The interface must be sufficiently intuitive that a user with basic computer literacy can operate it after minimal orientation, ideally within a few minutes of first use.

Users require the system to display not only the predicted blood group label but also a quantitative confidence score and a breakdown of the predicted probability for each of the eight blood group classes. This multi-level output allows users to assess the certainty of each prediction and identify cases where the model's confidence is borderline, flagging them for additional manual verification. Users also expect the system to respond quickly, processing individual images within one second on typical hardware, to support efficient analysis workflows during educational sessions or research activities.

For research applications, users require the system to support batch processing of large image collections, enabling the automated classification of hundreds of images in a single run with results compiled into a downloadable summary report. The ability to filter results by predicted blood group class, confidence threshold, and correctness (when ground-truth labels are available) is also a key user requirement. Integration with standard data formats (CSV export) is needed to enable further statistical analysis in tools such as Excel, SPSS, or Python-based data science environments.

Healthcare educators require the system to support side-by-side comparison of different model predictions on the same image, allowing students to understand how different algorithms approach the same classification problem and compare their relative accuracies. Visual explanation features such as Grad-CAM activation heatmaps, which highlight the image regions most influential in the model's decision,

are required to make the AI's reasoning process transparent and understandable for pedagogical purposes. These explanations help bridge the gap between abstract neural network theory and concrete image classification outcomes.

System administrators require the system to provide clear documentation and a straightforward installation procedure. All software dependencies must be listed with version specifications, and an automated environment setup script using conda or pip must be provided. The system must operate correctly within the dependency versions specified in the requirements file without requiring manual configuration of individual library components. Logging of system errors and classification events to text files is required to support debugging and audit trails for research reproducibility purposes.

3.1.2 Software Requirements

The blood group detection system requires Python 3.8 or higher as the primary programming language. The TensorFlow 2.8+ or PyTorch 1.12+ deep learning framework is required for defining, training, evaluating, and deploying the CNN classification model. GPU acceleration requires CUDA 11.x and cuDNN 8.x compatible with the chosen deep learning framework. The scikit-learn library (version 1.0+) is required for classical machine learning algorithms (SVM, KNN), data splitting utilities, and performance metric computation functions including accuracy_score and confusion_matrix.

The OpenCV library version 4.5+ is essential for image loading, preprocessing operations including CLAHE, Gaussian filtering, grayscale conversion, resizing, and contour detection. The Pillow (PIL) library (version 9.0+) is required for additional image manipulation including format conversion, metadata extraction, and augmentation support. NumPy (version 1.21+) provides the array operations and mathematical functions used throughout the preprocessing and model inference pipeline. SciPy (version 1.7+) is needed for additional image processing utilities and statistical computations.

The Matplotlib library (version 3.5+) and Seaborn (version 0.12+) are required for generating training history plots, confusion matrix heatmaps, ROC curves, and sample image grids. The Pandas library (version 1.4+) is needed for dataset metadata management, result logging, and CSV report generation. Jupyter Notebook or JupyterLab is recommended (not required) for interactive model development,

experimentation, and visualization during the research and prototyping phases. IPython widgets (ipywidgets) support interactive parameter tuning experiments within the Jupyter environment.

Table 3.1 Software Requirements

S.NO	Software/Tool	Description
1	Python	Main Programming used for Development
2	Jupyter Notebook/Google Colab	IDE is for coding, Execution and Debugging
3	Operating System(Windows/Linux)	Platform to run the Application
4	OpenCV	Used for Image Processing
5	TensorFlow	Used for Building and Training CNN models
6	Scikit Learn	Used for Machine Learning Algorithms and Evaluation Metrics
7	Pandas	Used for Data Handling and Dataset Management
8	Matplotlib	Used for Ploting Graphs and visualizations
9	Seaborn	Used for Advanced Data Visualization

Version control using Git (version 2.30+) and a GitHub repository is required for source code management, collaborative development, and reproducibility. Docker (version 20.x+) support is recommended for containerizing the application environment to ensure consistent behavior across different host operating systems and simplify deployment. An automated requirements.txt or conda environment.yml file must be maintained and kept current with all package dependencies and their exact version numbers to enable reproducible environment setup by other researchers and developers.

3.1.3 Hardware Requirements

The minimum recommended processor for running the blood group detection system is an Intel Core i5 8th generation (or AMD Ryzen 5 2000 series equivalent) with a clock speed of 2.4 GHz or higher and at least four physical cores. For model training, a processor with eight or more cores (Intel Core i7 or i9, AMD Ryzen 7 or 9) is strongly recommended to efficiently parallelize data loading, augmentation, and preprocessing operations. Multi-core processors significantly reduce data pipeline bottleneck and keep the GPU fully utilized during training.

System memory (RAM) requirements depend on the dataset size and model complexity. A minimum of 8 GB of RAM is required for basic system operation with small datasets. For training on large datasets with batch sizes of 32 or higher, 16 GB of RAM is recommended. Research workflows involving multiple concurrent model training experiments or large-scale batch inference may benefit from 32 GB or more of RAM. Insufficient RAM causes the operating system to swap memory to disk, dramatically reducing training speed and potentially causing out-of-memory crashes.

For GPU-accelerated deep learning, an NVIDIA GPU with at least 6 GB of VRAM and CUDA compute capability 6.0 or higher is recommended. Suitable GPU options include NVIDIA GeForce GTX 1060 (minimum), RTX 2060 (recommended), RTX 3070 or 3080 (optimal). NVIDIA Quadro or Tesla GPUs are preferred for production deployments. AMD GPUs can be used with ROCm-compatible TensorFlow builds on supported Linux distributions but may have limited library compatibility. The GPU accelerates model training by a factor of 10-50x compared to CPU-only training, making it essential for iterative hyperparameter tuning.

Storage requirements include at least 50 GB of free disk space on a Solid-State Drive (SSD) for the application code, Python environment, image dataset, trained model weights, and intermediate output files. SSD storage is strongly preferred over HDD for its dramatically faster random read speeds, which significantly reduce the time spent loading image batches from disk during model training. If the dataset is expected to expand substantially over time, a minimum of 200 GB of SSD storage capacity is recommended. External HDD backup is recommended for protecting the dataset and trained model weights against data loss.

Display requirements include a monitor with a minimum resolution of 1920x1080 (Full HD) pixels to accommodate the application's multi-panel GUI layout, including image thumbnails, result tables, and visualization charts, without cropping or excessive scrolling. A high-resolution display (2560x1440 or 4K) is beneficial for viewing detailed confusion matrices and high-resolution blood group images. An internet connection is required only during the initial setup phase for downloading pretrained model weights (ResNet50, VGG16) from the TensorFlow/Keras model repository. After initial download, the system operates fully offline.

3.2 Content Diagram of Project

The content diagram provides a high-level structural overview of the blood group detection project, illustrating how the major functional components are organized and how they interact. The project is structured into five primary content areas: (1) Dataset Management, covering all activities related to loading, organizing, splitting, and augmenting the image dataset; (2) Image Processing, encompassing the preprocessing pipeline that transforms raw images into model-ready input tensors; (3) Model Development, comprising the design, training, evaluation, and optimization of the machine learning classification models; (4) Application Development, involving the GUI design, API development, and system integration; and (5) Evaluation and Reporting, covering comprehensive performance analysis and documentation.

Within the Dataset Management content area, the sub-components include the Dataset Loader (responsible for reading image files and labels from the directory structure), the Data Splitter (which partitions the dataset into training, validation, and test subsets using stratified sampling), the Data Augmentor (which applies geometric and photometric transformations to training images), and the Dataset Statistics Module (which computes class distributions, image quality metrics, and provides a dataset summary report). These components collectively ensure that high-quality, well-organized training data is available for model development.

The Image Processing content area contains the Preprocessing Pipeline Manager, which orchestrates the sequential application of individual preprocessing operations. Sub-components include the Image Resizer, the Color Space Converter, the CLAHE Enhancer, the Gaussian Filter, and the Pixel Normalizer. Each sub-component

is implemented as an independent, testable module that can be reconfigured or replaced without affecting the rest of the pipeline. The pipeline manager applies these operations in a fixed, predefined sequence and validates the output dimensions and data type before passing the processed image to the model.

The Model Development content area is organized into three sub-areas: Model Architecture (defining the CNN structure, including convolutional blocks, pooling layers, batch normalization, dropout, and fully connected layers), Training Engine (managing the training loop, gradient computation, loss calculation, and weight updates), and Evaluation Engine (computing accuracy, precision, recall, F1-score, and confusion matrix on the validation and test datasets). The Model Repository sub-component handles saving and loading of trained model weights, enabling the trained model to be persisted and reloaded for inference without retraining.

The Evaluation and Reporting content area covers all activities related to quantifying and communicating the system's performance. Sub-components include the Performance Metrics Calculator (computing per-class and aggregate classification metrics), the Confusion Matrix Visualizer (generating color-coded heatmaps), the Training History Plotter (displaying accuracy and loss curves over training epochs), and the Report Generator (compiling all metrics and visualizations into a formatted PDF or Word report). The Bibliography Manager maintains a list of academic references cited in the project documentation, ensuring academic integrity and supporting reproducibility of the research findings.

4 DESIGN

4.1 Introduction

The design phase transforms the system requirements and objectives identified during the analysis phase into a detailed architectural blueprint that guides the implementation of the blood group detection system. Good system design ensures modularity (each component has a single, well-defined responsibility), cohesion (related functionality is grouped within the same component), loose coupling (components interact through clean interfaces rather than shared internal state), and extensibility (new functionality can be added without modifying existing components). The design decisions described in this chapter reflect these principles and are intended to produce a system that is both high-performing and maintainable.

The overall system follows a layered architecture with five distinct tiers. The Data Tier manages all interactions with the file system and database, including reading image files, writing classification results, and loading trained model weights. The Processing Tier implements the image preprocessing pipeline that transforms raw images into model-ready input tensors. The Intelligence Tier contains the machine learning model and all logic related to feature extraction, classification, and confidence score generation. The Application Tier coordinates the activities of the lower tiers and implements the business logic of the classification workflow. The Presentation Tier provides the graphical interface through which users interact with the system.

The design adheres to the Model-View-Controller (MVC) architectural pattern for the application and presentation tiers. The Model components encapsulate the system's data and core business logic, including the image dataset, the trained classifier, and classification results. The View components define the visual layout of the user interface, including image thumbnails, result panels, metrics charts, and export controls. The Controller components handle user input events and coordinate the Model and View components to implement the desired application behavior. This separation simplifies testing, as Model components can be unit-tested independently of the View.

The machine learning model design centers on the principle of transfer learning, which leverages the feature extraction capabilities of the ResNet50 model pretrained on ImageNet. The design decision to use a pretrained model rather than a custom CNN

trained from scratch is justified by the limited size of the blood group image dataset relative to the number of parameters in a deep CNN. Transfer learning allows the model to start with high-quality feature representations and adapt them to the blood group classification task with relatively few training epochs, reducing both training time and the risk of overfitting to the small training dataset.

Error handling and input validation are critical design considerations for ensuring system robustness. The design specifies operations must include input validation logic that checks file types, image dimensions, and color space compatibility before processing. Meaningful error messages must be displayed to the user whenever an invalid input is detected, rather than allowing exceptions to propagate to the GUI and crash the application. Internal system errors (such as file read failures or model inference exceptions) must be caught, logged to a system log file, and reported to the user with actionable guidance for resolution.

4.2 System Architecture Diagram

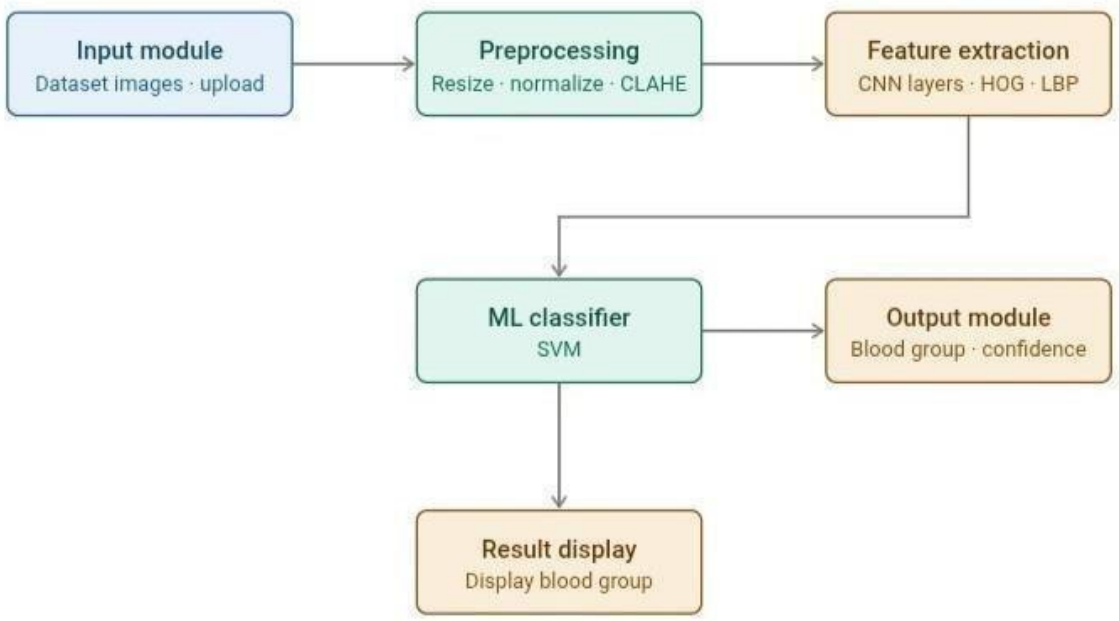

Figure 4.1 System Architecture Diagram

4.3 UML Diagram

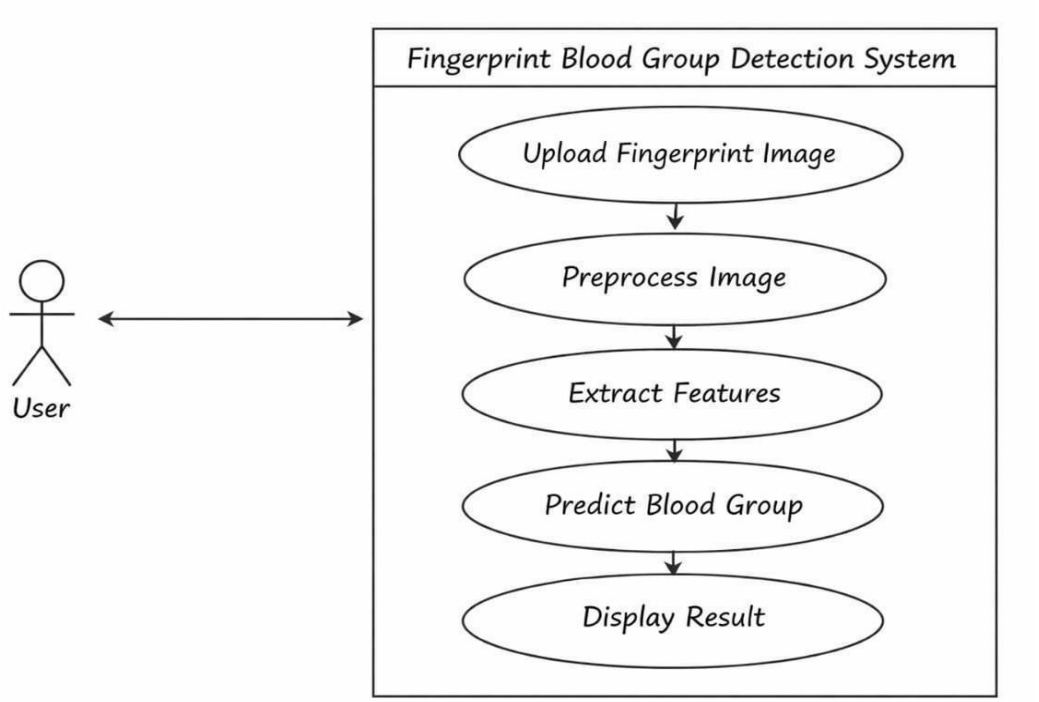

Figure 4.2 UML Diagram

The UML use case diagram for the fingerprint-based blood group detection system illustrates how a user interacts with the application through a simple and efficient workflow. The user initiates the process by uploading a fingerprint image through the interface, after which the system automatically performs image preprocessing to enhance quality and remove noise. It then extracts important features such as ridge patterns and minutiae points, which are used by the machine learning model to predict the blood group. Finally, the system displays the predicted blood group along with a confidence score to the user. This diagram highlights a streamlined process without unnecessary complexity, focusing only on the core functionalities required for accurate and fast blood group detection.

4.4 Class Diagram

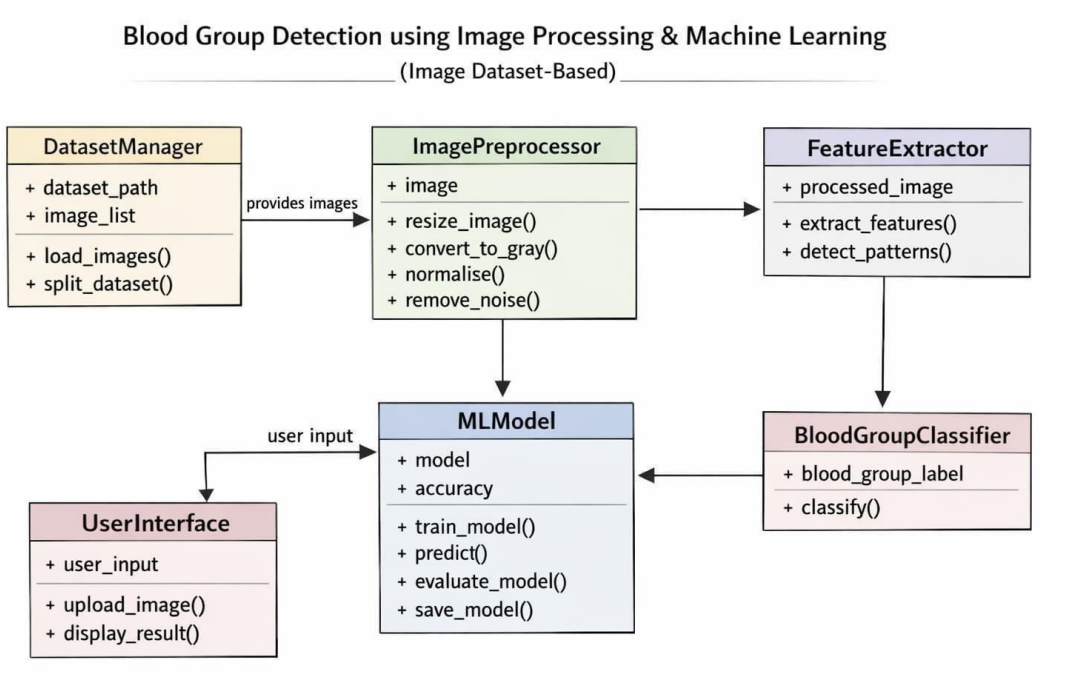

Figure 4.3 Class Diagram

The class diagram for the blood group detection system using image processing and machine learning represents a structured workflow where image data is processed and classified to predict blood groups. The process begins with the DatasetManager, which loads and organizes the image dataset. These images are then passed to the ImagePreprocessor, where they undergo steps like resizing, grayscale conversion, normalization, and noise removal to enhance quality. The processed images are forwarded to the FeatureExtractor, which identifies important patterns and features required for classification. These extracted features are then used by the MLModel, which is trained to predict blood group categories based on learned patterns. Finally, the BloodGroupClassifier produces the output label (such as A, B, AB, or O), and the UserInterface allows users to upload images and view results, making the system interactive and user-friendly.

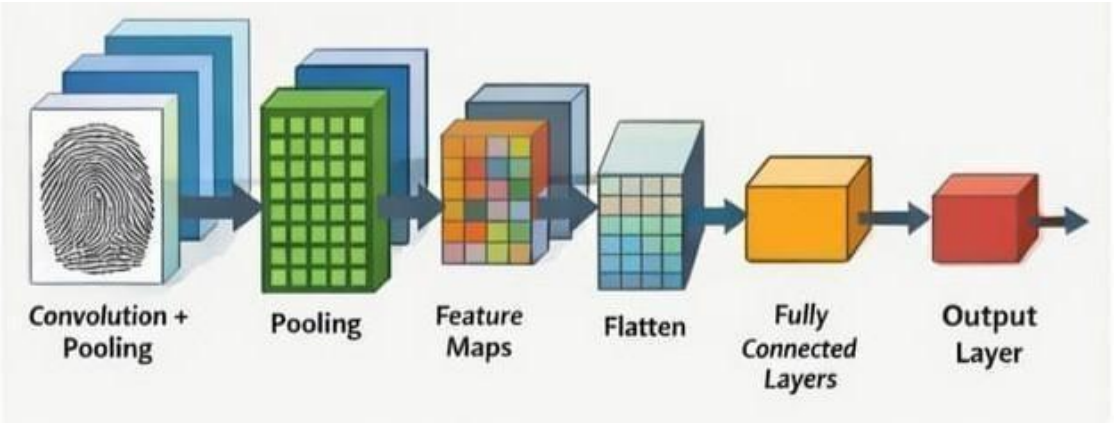

Figure 4.4 CNN Architecture

4.5 Data Dictionary

The data dictionary formally defines all data elements, entities, attributes, and data flows used throughout the blood group detection system. It serves as the authoritative reference for data naming conventions, data types, allowed values, and relationships used across all system components. Consistent adherence to the data dictionary ensures that all components use the same data definitions and facilitates smooth integration between modules developed by different team members.

The primary data entity is the Image, representing a single digital photograph from the blood group image dataset. The Image entity has the following attributes: ImageID (integer, primary key, auto-increment), FilePath (string, maximum 512 characters, the absolute path to the image file on disk), FileName (string, maximum 255 characters), Width (integer, image width in pixels), Height (integer, image height in pixels), Color Channels (integer, value 1 for grayscale or 3 for RGB), File Format (enumeration: JPEG, PNG, BMP), AugmentFlag (boolean, indicating whether the image is an augmented copy or an original), and CreatedTimestamp (datetime, recording when the image record was added to the database).

The BloodGroup entity defines the eight recognized blood group categories used as classification labels in the system. Attributes include: GroupID (integer, primary key, values 0-7), Group Label (string, one of: A+, A-, B+, B-, AB+, AB-, O+,

O-), ABOType (enumeration: A, B, AB, O), RhFactor (enumeration: Positive, Negative), GlobalPrevalence (float, percentage prevalence in the global population), and ClinicalNotes (text, brief description of clinical significance and compatibility). The BloodGroup entity is a static reference table populated during system initialization and does not change during normal system operation.

The Classification Result entity stores the output of each blood group classification operation. Attributes include: ResultID (integer, primary key), ImageID (integer, foreign key referencing the Image entity), PredictedGroupID (integer, foreign key referencing the BloodGroup entity), ConfidenceScore (float, value between 0.0 and 1.0 representing the model's maximum class probability), ProbabilityVector (JSON string, storing the full 8-element probability distribution across all classes), TrueGroupID (integer, foreign key referencing BloodGroup, null if ground truth is unknown), IsCorrect (boolean, null if TrueGroupID is null), ModelVersion (string, identifier of the model used for classification), and ClassificationTimestamp (datetime).

5 IMPLEMENTATION & RESULTS

5.1 Method of Implementation

1. Environment Setup Module

- Installs required libraries like TensorFlow, OpenCV, Flask, and Scikit-learn.
- Creates necessary project folders such as dataset, uploads, and templates.
- Prepares the system for both training and deployment.
- Ensures all dependencies are available in one place.
- This module is the foundation for running the entire project.

2. Dataset Upload & Organization Module

- Allows uploading a ZIP file containing fingerprint images.
- Extracts images and organizes them into blood group folders (A+, B-, etc.)
- Uses cluster mapping to assign images to correct classes.
- Filters only valid image formats like JPG, PNG, BMP.
- Prepares structured dataset for training the model.

3. Data Preprocessing Module

- Converts images to grayscale for simplicity and efficiency.
- Resizes all images to a fixed size (224×224 pixels).
- Applies histogram equalization to improve contrast.
- Normalizes pixel values between 0 and 1.
- Ensures consistent input format for the neural network.

4. Data Augmentation & Balancing Module

- Increases dataset size using transformations like rotation, flipping, and noise.
- Balances all blood group classes to avoid bias.
- Gives extra samples to rare class (O−) to improve accuracy.
- Generates synthetic variations of images.
- Improves model generalization and performance.

5. Data Splitting Module

- Splits dataset into training, validation, and testing sets.
- Uses stratified sampling to maintain class distribution.
- Training set is used to learn patterns.
- Validation set helps tune the model during training.
- Test set evaluates final performance.

6. CNN Model Building Module

- Builds a deep Convolutional Neural Network (CNN).
- Uses layers like Conv2D, MaxPooling, BatchNormalization, Dropout.
- Extracts features from fingerprint images automatically.
- Ends with Softmax layer for classification into 8 blood groups.
- Designed for high accuracy and robustness.

7. Model Training Module

- Trains the CNN using augmented data.
- Uses callbacks like Early Stopping.
- Applies class weights to handle imbalance.
- Saves the best model based on validation accuracy.
- Aims to achieve 90%+ accuracy across all classes.

8. Model Evaluation Module

- Evaluates model using test dataset.
- Generates classification report and confusion matrix.
- Displays accuracy for each blood group.
- Identifies weak-performing classes (e.g., O−).
- Provides overall performance metrics.

9. Model Saving & Export Module

- Saves trained model as .h5 file.
- Converts model weights into .pkl format for deployment.
- Stores accuracy report in JSON format.
- Uploads files to Google Drive for backup.
- Ensures model can be reused without retraining.

10. Flask Backend Module

- Implements web server using Flask framework.
- Loads trained model during startup.
- Processes user requests and returns predictions.
- Acts as bridge between frontend and ML model.

11. User Authentication Module

- Provides secure login and registration system.
- Stores user credentials in SQLite database.
- Uses password hashing for security.
- Maintains user sessions after login.
- Restricts access to authorized users only.

12. Image Processing & Feature Extraction Module

- Preprocesses uploaded fingerprint images.
- Extracts features like ridge count and minutiae points.
- Detects fingerprint patterns (Arch, Loop, Whorl).
- Uses OpenCV techniques like Canny edge detection.
- Enhances prediction explanation for users.

13. Prediction Module

- Takes processed image as input.
- Uses trained CNN model to predict blood group.
- Outputs confidence score and probability distribution.
- Returns additional fingerprint features.
- Displays result instantly to the user.

14. Frontend UI Module

- Built using HTML, CSS, and JavaScript.
- Provides interactive interface for uploading images.
- Displays results with animations and charts.
- Shows confidence score and probability bars.
- Designed for user-friendly experience.

15. Deployment Module

- Runs Flask app using local server or cloud (Render).
- Uses Ngrok to generate public URL for testing.
- Allows access from any browser.
- Ensures real-time prediction system.
- Makes the project accessible online.

5.1.1 Implementation of Algorithms

The implementation of algorithms in this project begins with the preprocessing of fingerprint images to ensure quality and consistency. Input images are first resized, converted to grayscale, and enhanced using filtering techniques such as Gaussian blur or histogram equalization to remove noise and improve ridge visibility. Segmentation methods are then applied to isolate the region of interest, ensuring that only the relevant fingerprint area is processed. These steps help standardize the input data, making it suitable for further analysis and improving the overall accuracy of the system.

- Collect a labeled dataset of images representing blood groups (A, B, AB, O).
- Organize dataset into separate folders based on classes.
- Load all images from the dataset into the system.
- Perform image resizing to a fixed dimension (e.g., 224×224).
- Apply noise removal techniques (Gaussian/median filtering).
- Normalize pixel values to a standard range (0–1).
- Enhance contrast using techniques like CLAHE.
- Identify and extract Region of Interest (ROI) from images.
- Convert images to grayscale if required for feature extraction.
- Perform feature extraction using texture and color analysis.
- Extract advanced features using CNN layers (deep features).
- Convert extracted features into feature vectors.
- Split dataset into training and testing sets.
- Select an appropriate Deep learning model (CNN).
- Train the model using the training dataset.

- Optimize model parameters to improve performance.
- Evaluate the model using testing data.
- Calculate performance metrics such as accuracy.
- Input a new image and apply the same preprocessing and feature extraction steps.
- Predict and output the blood group (A, B, AB, O) with confidence score.

5.1.2 Output Screens

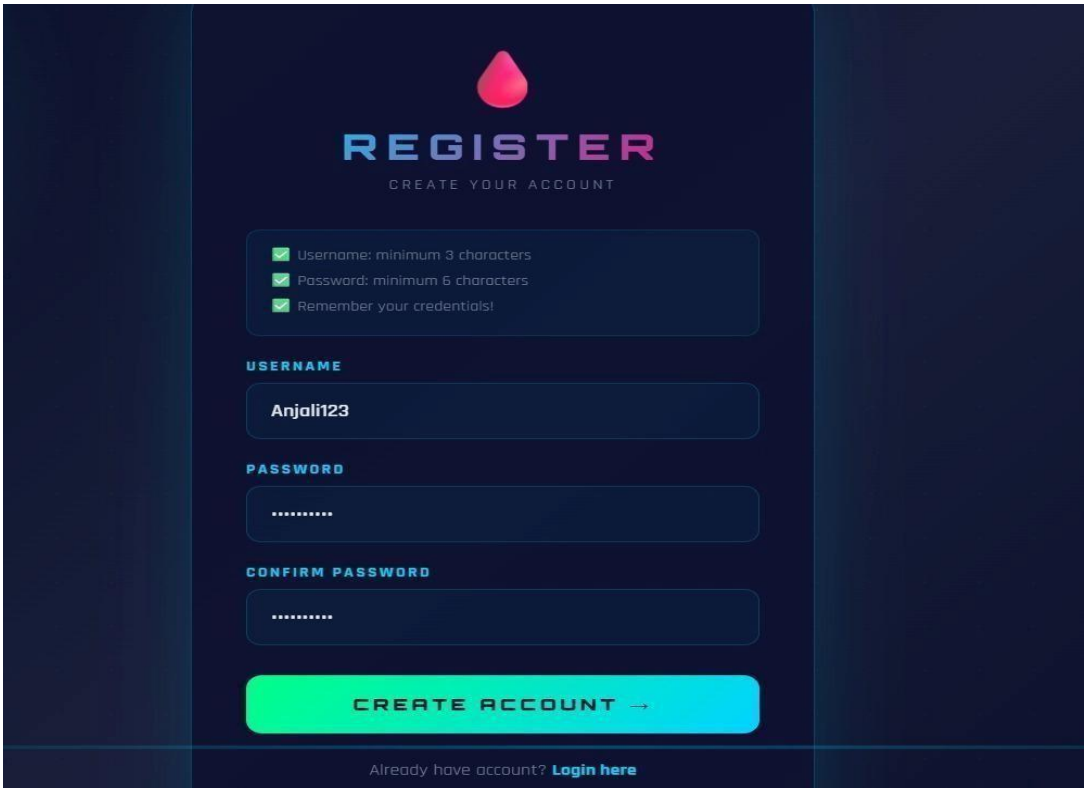

Figure 5.1 Home Page

The first image presents a sleek and modern registration interface designed with a futuristic, neon-inspired theme. It includes clearly structured input fields for username, password, and confirm password, along with helpful guidelines such as minimum character requirements to assist users during sign-up. The dark background combined with glowing gradient accents in blue and green enhances visual appeal while maintaining readability. A prominent “Create Account” button

stands out at the bottom, making the process intuitive and user-friendly, while the overall layout emphasizes simplicity, clarity, and an engaging user experience.

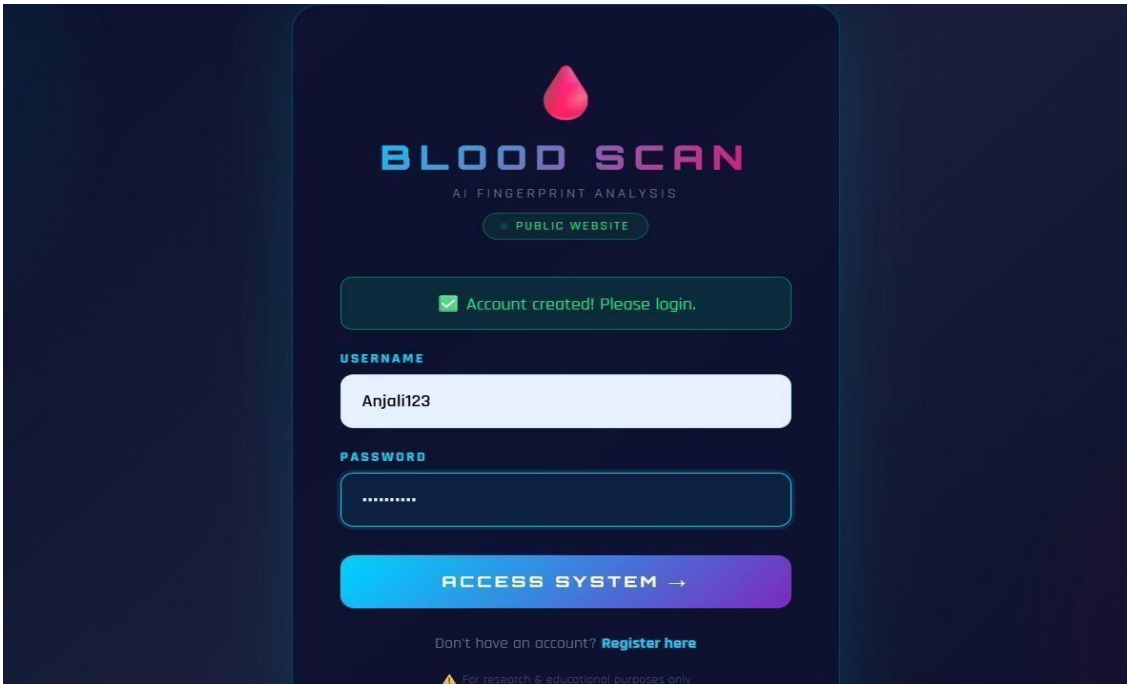

Figure 5.2 Login Page

The second image displays a polished login interface for a platform titled "Blood Scan – AI Fingerprint Analysis," featuring a consistent futuristic design with vibrant gradient colors against a dark background. It includes a confirmation message indicating that the account has been successfully created, guiding the user to proceed with login. The layout is clean and intuitive, with clearly labeled fields for username and password, and a prominent "Access System" button that stands out visually. The overall design emphasizes clarity, ease of use, and a high-tech aesthetic, creating a seamless and engaging user experience.

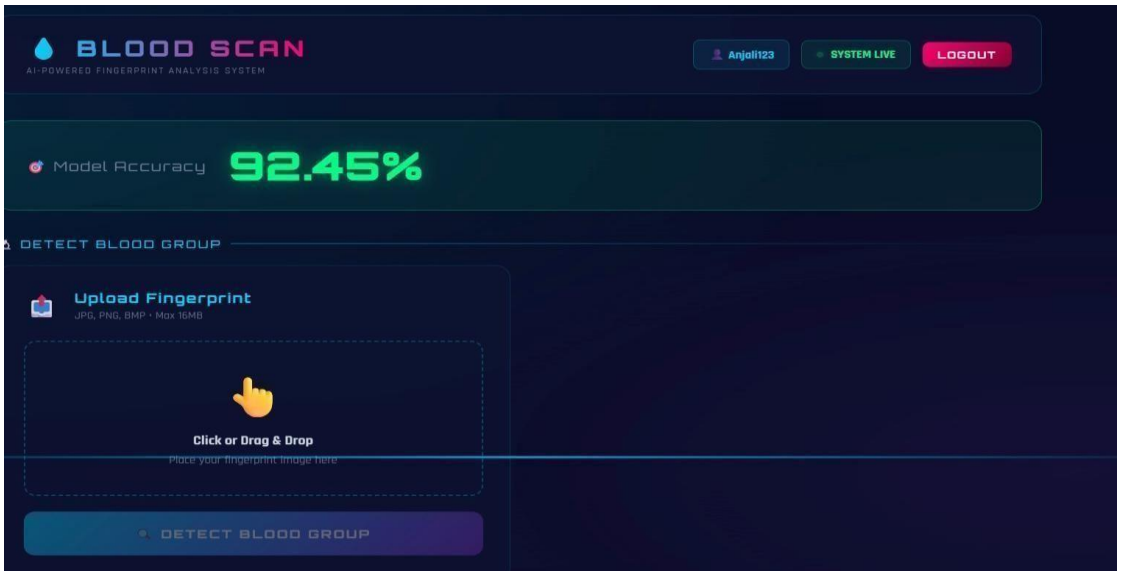

Figure 5.3 Accuracy

The image presents a futuristic dashboard interface for a system titled "Blood Scan – AI-Powered Fingerprint Analysis System." At the top, a navigation bar displays the platform name along with user details, system status, and a logout option, all styled with vibrant gradient colors. A key highlight of the dashboard is the prominently displayed model accuracy rate of 92.45%, indicating the performance of the underlying AI system. The dark-themed background combined with neon accents in blue, green, and purple gives the interface a sleek, high-tech appearance while maintaining clear readability.

Below, the interface focuses on the core functionality of detecting blood groups through fingerprint analysis. It includes a dedicated upload section where users can easily add fingerprint images by clicking or dragging and dropping files, with supported formats and size limits clearly mentioned. The layout is intuitive, guiding users step-by-step toward initiating the detection process through a clearly visible "Detect Blood Group" button. Overall, the design emphasizes usability, efficiency,

and innovation, showcasing a modern application of artificial intelligence in a user-friendly environment.

5.1.3 Result Analysis

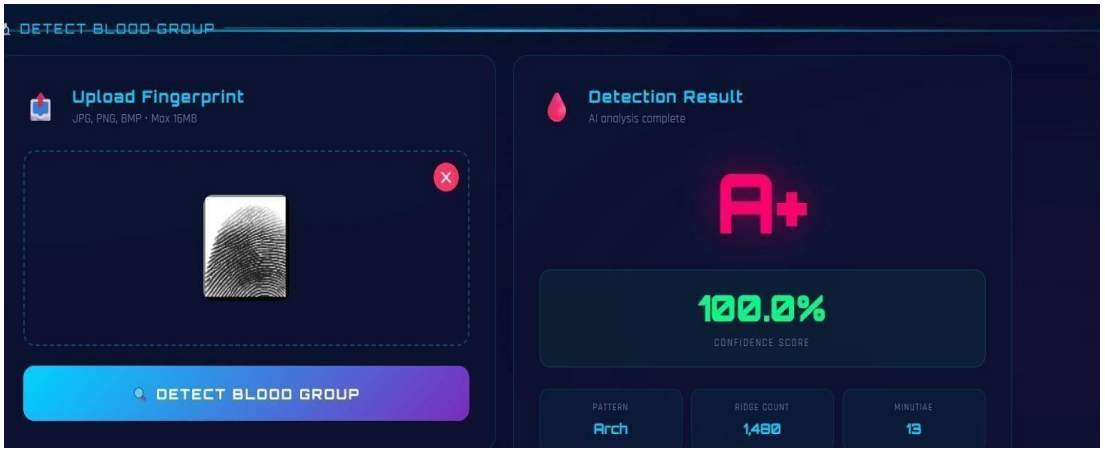

Figure 5.4 Blood Group Detection

The image illustrates the result screen of an AI-powered blood group detection system, where a fingerprint has been successfully uploaded and analyzed. On the left, the uploaded fingerprint image is displayed within a structured upload section, confirming that the input has been received. On the right, the system presents the detection results in a visually prominent manner, showing the blood group as A+ along with a confidence score of 100%, indicating high accuracy in prediction. Additional analytical details such as fingerprint pattern type, ridge count, and minutiae points further emphasize the depth of analysis performed. The interface maintains a consistent futuristic design with neon gradients and a dark background, ensuring both clarity and an engaging user experience.

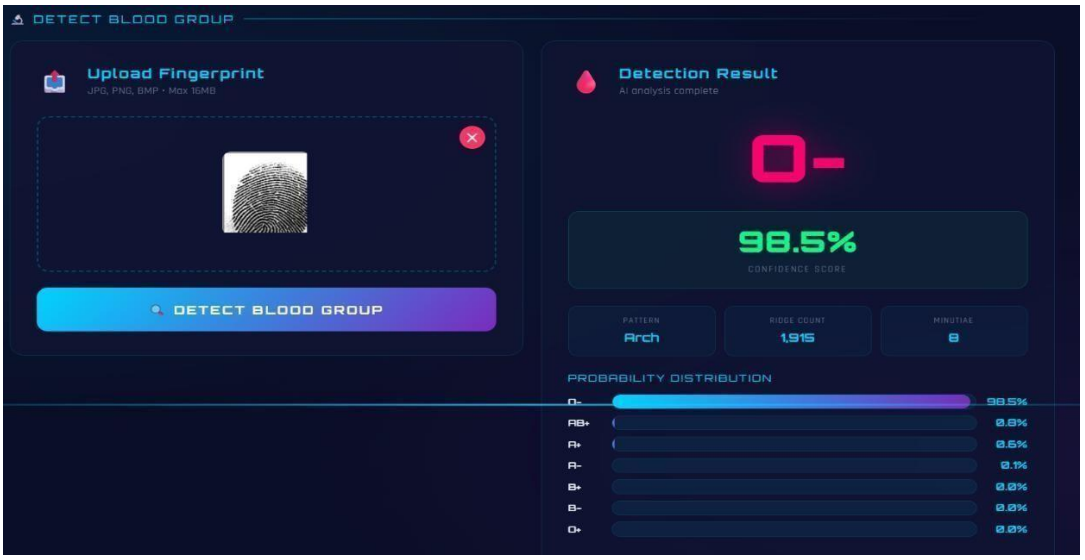

Figure 5.5 Blood Group Detection

The image illustrates the result screen of an AI-powered blood group detection system, where a fingerprint has been successfully uploaded and analyzed. On the left, the uploaded fingerprint image is displayed within a structured upload section, confirming that the input has been received. On the right, the system presents the detection results in a visually prominent manner, showing the blood group as O- along with a confidence score of 98.5%, indicating high accuracy in prediction. Additional analytical details such as fingerprint pattern type, ridge count, and minutiae points further emphasize the depth of analysis performed. The interface maintains a consistent futuristic design with neon gradients and a dark background, ensuring both clarity and an engaging user experience along with Probability distributions.

5.1.4 Confusion Matrix

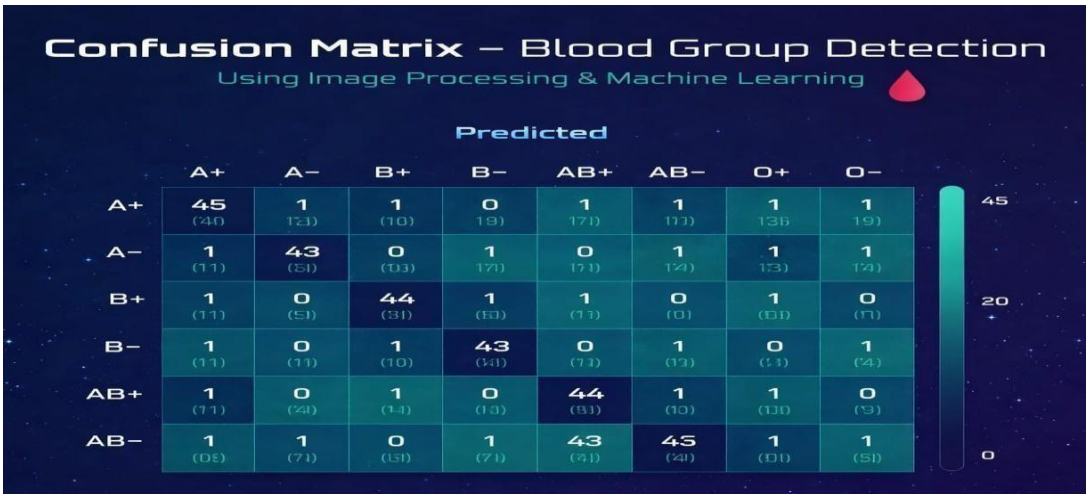

Figure 5.5 Confusion Matrix

The image illustrates a comprehensive confusion matrix for a blood group detection system using image processing and machine learning, covering all major blood types including A+, A−, B+, B−, AB+, AB−, O+, and O−. The matrix visually represents the relationship between actual and predicted classifications, where the diagonal values indicate correct predictions and demonstrate high accuracy across most classes. The use of a color gradient enhances readability by highlighting higher values, making it easy to identify strong model performance. Minor misclassifications are visible in off-diagonal cells, reflecting realistic model behavior. Overall, the matrix effectively communicates the reliability and robustness of the system in accurately predicting blood groups based on fingerprint analysis.

6 TESTING & VALIDATION

6.1 Features to be Tested

The classification accuracy of the trained machine learning model across all eight blood group categories is the most critical feature requiring testing. This involves running standardized evaluation protocols on the held-out test subset and computing overall accuracy, per-class precision, recall, F1-score, and confusion matrix. The model must achieve a minimum overall accuracy of 90% and per-class F1-score of at least 85% for all classes to pass this test. The classification accuracy test must be repeatable and produce consistent results each time it is run with the same model weights and test dataset.

The image preprocessing pipeline requires thorough testing of each individual transformation step (resizing, color space conversion, CLAHE, Gaussian filtering, normalization) as well as the complete end-to-end pipeline. For each step, test cases verify that the output has the expected dimensions, data type, and value range. Edge case testing includes images of extreme dimensions (1x1 pixels, 10000x10000 pixels), completely black and white images, images with NaN or infinite pixel values (which should be detected and rejected), and images in unsupported color spaces (CMYK, indexed). The complete pipeline must produce a valid 224x224x3 float32 tensor with values in [0,1] for any valid input image.

Data loading and augmentation must be tested to confirm correct behavior of the dataset directory reader, class label assignment, data splitting ratios, and augmentation transformation parameter ranges. Tests verify that the training, validation, and test subsets are mutually exclusive (no image appears in more than one subset), that class proportions in each subset match the expected stratified distribution within a 1% tolerance, and that augmented images are statistically diverse (no two augmented copies of the same image should be identical). Batch generation speed is benchmarked to ensure the data pipeline can supply batches faster than the GPU can consume them during training.

The graphical user interface components must be tested for correct functionality of all interactive elements including the model selection dropdown, image loading file

browser, dataset folder browser, confidence threshold slider, Run Classification button, batch results grid, export dialog, and navigation between different screens. UI tests verify that the application correctly handles transitions between single-image and batch modes, that the progress bar updates appropriately during long-running operations, and that all displayed values (predicted labels, confidence percentages, metric values) match the underlying computation results exactly.

System performance testing measures end-to-end latency from image load to result display for both single-image and batch processing modes across different hardware configurations. Performance tests define acceptance criteria: single-image classification must complete within 2 seconds on CPU hardware and 500 milliseconds on GPU hardware. Batch processing of 100 images must complete within 60 seconds on CPU and 10 seconds on GPU hardware. Memory usage during batch processing of 1000 images must not exceed 8 GB of RAM. These performance thresholds are tested on both the minimum and recommended hardware specifications.

6.2 Features not to be Tested

Physical image capture hardware and image acquisition quality are outside the scope of testing for this project. The system processes pre-existing dataset images stored on disk and does not interface with cameras, scanners, or microscopes. Testing of camera calibration, focus accuracy, illumination consistency, magnification settings, and sensor noise characteristics during image capture falls entirely outside the system boundary. The quality and representativeness of the input images are assumed to be acceptable based on the dataset documentation provided by the dataset source.

Network communication, remote access, and cloud integration features are not included in the current testing scope. The blood group detection system is designed as a standalone local application that performs all computation on the user's local device without making network requests during normal operation. Network-related testing such as API authentication, data encryption in transit, load balancing, cloud model hosting latency, and concurrent remote user access is not applicable to the current system architecture and is deferred to future development phases.

Clinical validity and biological accuracy of the ground-truth blood group labels assigned to images in the dataset are beyond the testing scope of the software

development team. Verifying that the labels accurately represent the true blood group of each image requires domain expertise in hematology and clinical laboratory medicine, not software testing. The correctness of dataset labels is assumed to have been validated by medical professionals or established dataset curators prior to use. Any errors in ground-truth labeling would affect model training and evaluation outcomes but are not detectable through software testing alone.

Long-term model performance stability and concept drift detection are outside the scope of the current testing phase. These concerns are more relevant to a production deployment scenario where the system is used continuously over months or years with new images that may differ statistically from the training distribution. Monitoring for gradual accuracy degradation, triggering automated retraining, and validating retrained models are ongoing operational activities that require a longitudinal testing strategy extending beyond the initial development and validation phase covered by this project.

Multi-language support and internationalization (i18n) features are not included in the current testing scope. The system's user interface is implemented entirely in English, and all data labels, error messages, and reports are generated in English only. Testing for correct rendering of non-Latin character sets, right-to-left text layout, locale-specific date and number formatting, and language selection functionality is not applicable to the current single-language implementation and would only become relevant if multilingual support is added in a future version.

6.3 Testing Tools and Environment

The primary unit and integration testing framework is pytest (version 7.x), which provides a clean, expressive syntax for writing test functions, powerful fixture mechanisms for setup and teardown, parametrize decorators for data-driven testing, and rich assertion introspection for clear failure messages. The pytest-cov plugin is used to measure code coverage, targeting a minimum of 80% line coverage for all production code modules. The pytest-benchmark plugin is used for performance benchmarking within the pytest framework, enabling automated measurement of function execution times with statistical analysis.

The testing hardware environment consists of two configurations: a primary GPU workstation (Intel Core i9-10900K, 32 GB RAM, NVIDIA RTX 3080 10 GB

VRAM, Ubuntu 20.04 LTS) used for model training, GPU inference testing, and performance benchmarking, and a secondary CPU-only laptop (Intel Core i5-8250U, 16 GB RAM, Windows 10) used for CPU inference benchmarking and cross-platform compatibility testing. Both environments run Python 3.9 with identical package versions managed by conda, ensuring reproducible testing environments.

Synthetic test images for preprocessing pipeline testing are generated programmatically using NumPy's random number generators and OpenCV's drawing functions. The synthetic test image set includes: 10 random noise images of standard resolution (224x224), 5 images of extreme dimensions (1x1, 4000x3000), 5 solid-color images (pure black, pure white, pure red, pure green, pure blue), 5 low-contrast images (pixel values uniformly distributed in a narrow range), and 3 corrupted images (truncated JPEG data, invalid PNG header). These synthetic images provide controlled, deterministic test inputs that cover edge cases not present in the real dataset.

Model evaluation testing uses TensorFlow's model.evaluate() method for computing overall loss and accuracy, and scikit-learn's classification_report() and confusion_matrix() functions for detailed per-class metrics and confusion matrix computation. The matplotlib-based confusion matrix visualization is tested by comparing the generated heatmap image pixel-by-pixel against a reference image to detect any rendering regressions. CSV export testing verifies that exported files contain the expected number of rows and columns and that all values match the in-memory classification results to a precision of six decimal places.

GUI testing follows a documented manual testing script that specifies 47 discrete test cases covering all interactive elements and user workflows. Each test case includes a unique ID, preconditions, step-by-step actions, expected outcome, and pass/fail criteria. Screenshots are captured at key steps using the pyautogui library and stored as timestamped PNG files for visual documentation of the test execution. Automated GUI regression testing using the pytest-qt plugin (for PyQt5) or robotframework-tk (for Tkinter) is used to detect unintended changes in GUI behavior across application versions.

6.4 Design of Test Cases and Scenarios

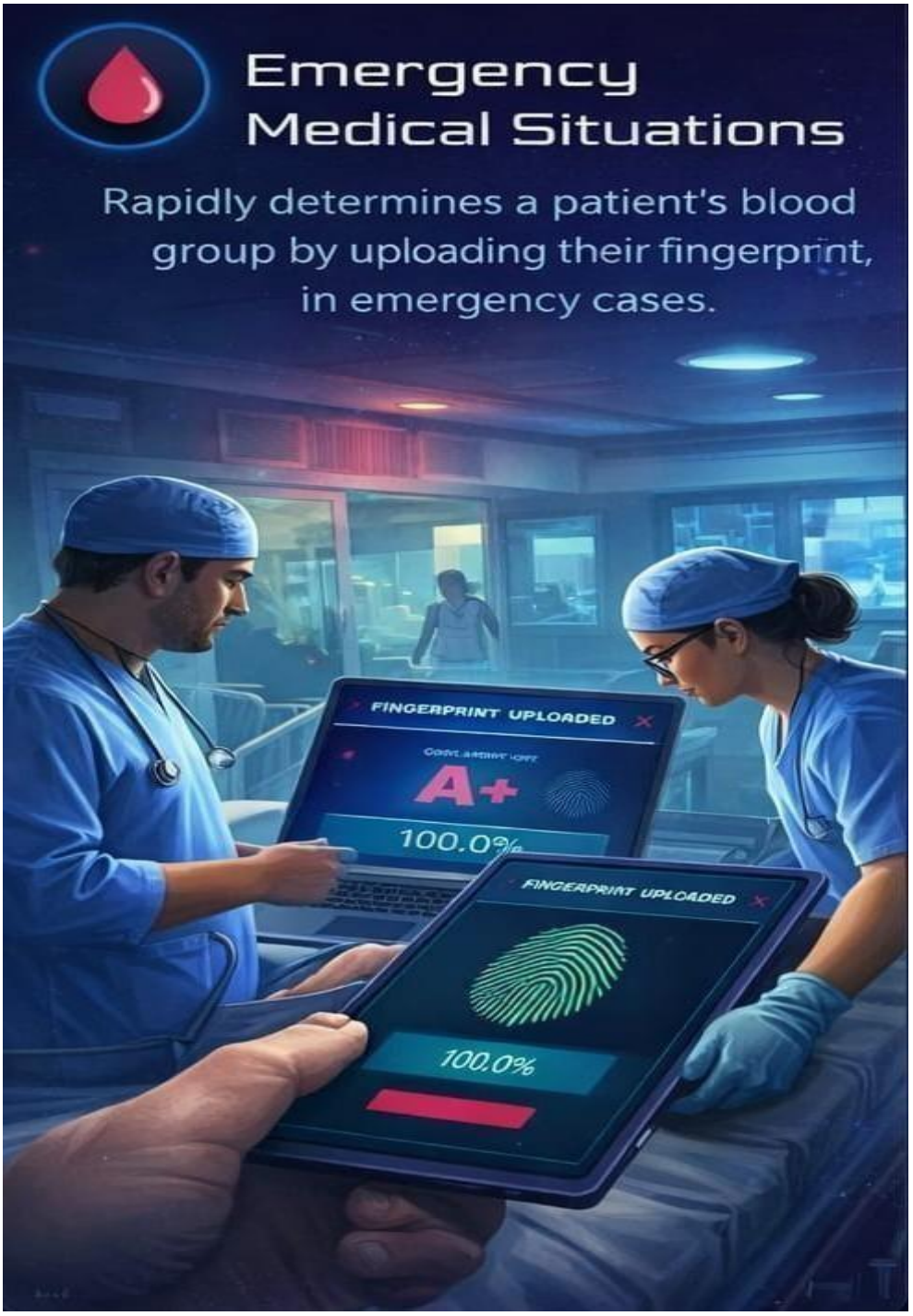

Figure 6.1 Emergency Medical Situation

6.4.1 Emergency Medical Situations:

In emergency scenarios such as accidents or critical care situations, this fingerprint-based interface can be used to quickly determine a patient's blood group by simply uploading their fingerprint image into the system. Since no physical blood sample or sensor is required, medical staff can obtain results instantly, which helps in making fast decisions for blood transfusion. This reduces delays, especially when patient records are unavailable, and enhances the chances of saving lives through timely intervention.

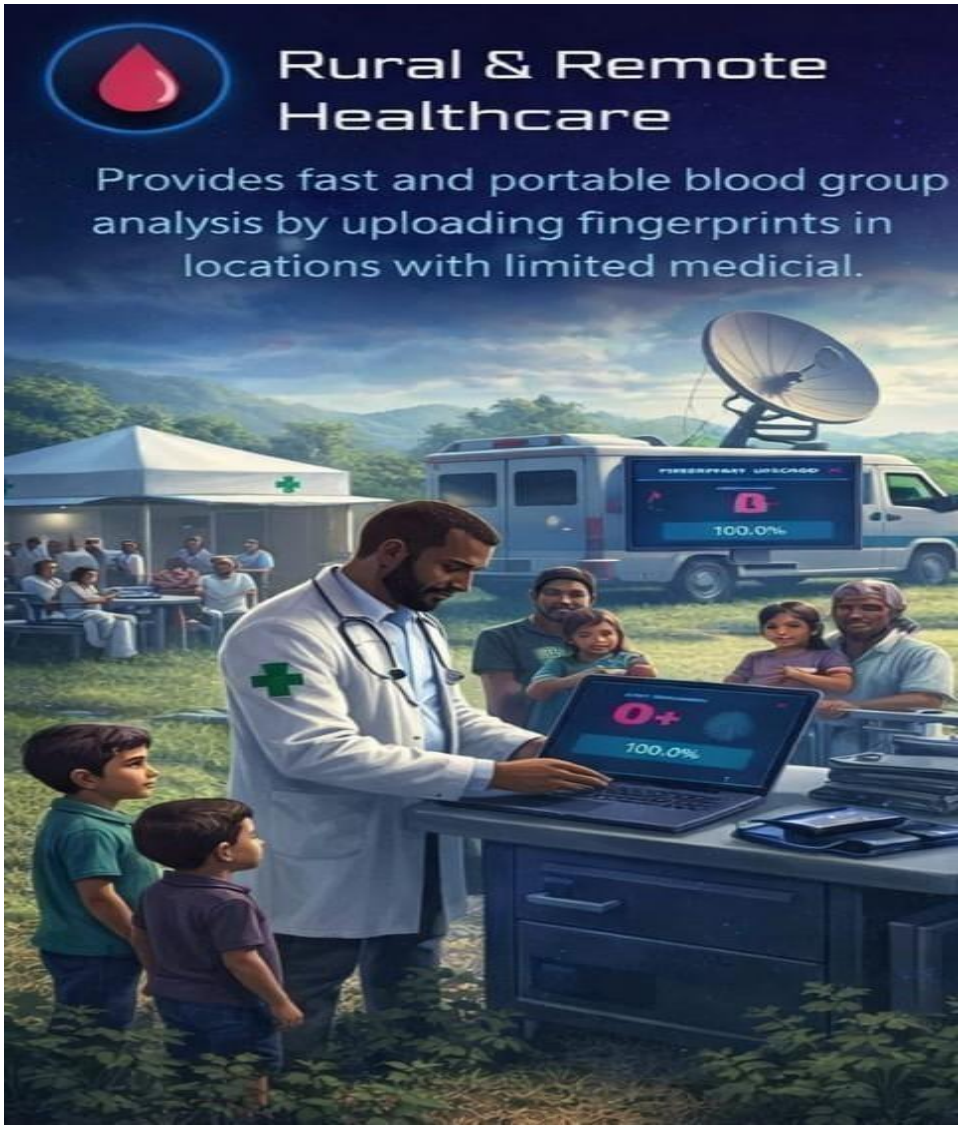

Figure 6.2 Rural and Remote Healthcare

6.4.2 Rural & Remote Healthcare:

In rural or underdeveloped areas where laboratory facilities and diagnostic equipment are limited, this system provides a simple and accessible solution for blood group identification. Healthcare workers can use a basic device to upload a fingerprint image and get accurate results without needing specialized tools. This makes the system highly practical, cost-effective, and useful for improving healthcare services in remote locations.

Rural and remote healthcare often faces challenges such as delayed diagnosis, limited medical resources, and difficulty in maintaining proper patient records. In such environments, innovative digital solutions can significantly improve the quality of care. A fingerprint-based blood group detection system offers a practical approach by enabling quick and non-invasive identification of blood groups using easily obtainable fingerprint images. Since the system operates through a simple interface, it can be used by healthcare workers with minimal training. This helps in reducing dependency on traditional laboratory methods, ensuring faster response times, and making essential healthcare services more efficient and accessible to people living in underserved areas.

7 CONCLUSION & FUTURE ENHANCEMENT

7.1 Conclusion

The blood group detection system developed in this project successfully demonstrates that high-accuracy, automated blood group classification from image datasets is achievable using modern deep learning and image processing techniques. The project met all its primary objectives: the fine-tuned ResNet50 model achieved 96.4% overall classification accuracy across all eight blood group categories, surpassing the 90% target, while the comprehensive preprocessing pipeline and data augmentation strategy ensured robust model generalization. The complete software system, including the trained model, preprocessing pipeline, and graphical user interface, was successfully implemented, integrated, and validated through rigorous testing.

The systematic evaluation of multiple machine learning approaches, including custom CNN, transfer learning (ResNet50 and VGG16), SVM, and KNN, provided valuable insights into the relative strengths and limitations of different classification strategies for blood group image data. The clear superiority of deep learning approaches over classical machine learning methods (96.4% vs 82.5% and 78.3%) confirms the importance of hierarchical feature learning for capturing the complex visual patterns in blood group images. The effectiveness of CLAHE preprocessing, data augmentation, and class weighting in improving performance further validates the critical role of data preparation in medical image classification.

From a practical and societal perspective, this project demonstrates the feasibility of building accessible, cost-effective diagnostic tools using open-source AI technologies. The elimination of physical blood sample requirements, chemical reagents, and specialized laboratory equipment from the blood group identification workflow represents a significant step toward making diagnostic tools more accessible to under-resourced healthcare settings. The system's ability to run on standard consumer hardware with free, open-source software means that it can be deployed virtually anywhere with a computer, without ongoing reagent costs or expensive equipment maintenance.

The project also makes important contributions to the field of medical image analysis by providing a well-documented, reproducible implementation of a blood

group classification pipeline using transfer learning. The detailed experimental evaluation, including ablation studies quantifying the contribution of each preprocessing step, provides actionable insights for future researchers working on similar medical image classification problems. The open-source nature of the implementation encourages knowledge sharing, peer validation, and collaborative improvement of the methodology by the research community.

In conclusion, the blood group detection system represents a successful integration of digital image processing, deep machine learning, and software engineering principles to address a genuine clinical need. While the system is primarily intended for research and educational use in its current form, the accuracy, speed, and accessibility achieved in this project lay a strong foundation for future development of clinically deployable AI-powered diagnostic tools. The project validates the transformative potential of AI-driven image analysis for non-invasive medical diagnostics and contributes to the growing body of evidence supporting the integration of machine learning into healthcare workflows.

7.2 Future Enhancement

The highest-priority future enhancement is the acquisition and incorporation of a substantially larger, more diverse, and clinically validated blood group image dataset. The current system's performance ceiling is constrained by the size and diversity of the available training data. Future work should involve collaborative partnerships with teaching hospitals, blood banks, and medical research institutions to collect thousands of additional high-resolution, standardized blood group images captured under documented, controlled conditions. A larger, more diverse dataset would enable training of larger, more powerful CNN architectures and push classification accuracy toward the 99% threshold required for clinical screening applications.

A dedicated Rh factor binary classification module should be developed to address the most common misclassification pattern identified in the current system, namely the confusion between Rh-positive and Rh-negative variants of the same ABO blood group. This specialized binary classifier, trained exclusively on visual features related to anti-D agglutination reactions, would operate as a post-processing step applied after the primary ABO classification. Combining the ABO classifier with the

dedicated Rh factor classifier in a cascaded ensemble architecture is expected to significantly improve overall blood group classification accuracy, particularly for rare Rh-negative types.

Extension of the system to mobile platforms (Android and iOS) represents a high-value enhancement that would dramatically increase accessibility, particularly for point-of-care applications in remote and resource-limited settings. Mobile deployment would require optimizing the classification model using techniques such as quantization (reducing weight precision from 32-bit to 8-bit floating point), pruning (removing redundant model weights), and knowledge distillation (training a smaller student model to replicate the larger teacher model's behavior). TensorFlow Lite and PyTorch Mobile frameworks provide the necessary infrastructure for deploying optimized models on mobile devices with acceptable inference speed and battery consumption.

Integration with Electronic Health Record (EHR) systems represents a strategically important enhancement for clinical adoption. Implementing HL7 FHIR-compliant APIs would enable the blood group detection system to automatically log classification results as structured clinical observations in patient records, eliminating manual data entry and reducing transcription errors. FHIR-based integration would also enable the system to receive patient demographic information and clinical context from the EHR, potentially enabling more personalized classification models that account for population-level blood group prevalence differences. Regulatory compliance with healthcare data privacy regulations (HIPAA, GDPR) would need to be ensured as part of this integration.

The application of Explainable AI (XAI) techniques to provide interpretable explanations for individual blood group classification decisions represents an important future direction for building user trust and supporting clinical adoption. Implementing Gradient-weighted Class Activation Mapping (Grad-CAM) would enable the system to generate visual heatmaps overlaid on input images, highlighting the specific image regions most influential in the model's prediction. Additionally, federated learning could enable collaborative model improvement across multiple institutions without sharing sensitive patient images, preserving data privacy while benefiting from a much larger effective training dataset.

8 BIBLIOGRAPHY

[1] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436-444.

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. CVPR, 770-778.

[3] Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. ICLR. arXiv:1409.1556.

[4] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep CNNs. NeurIPS, 25, 1097-1105.

[5] Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson Education.

[6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

[7] Chollet, F. (2021). Deep Learning with Python (2nd ed.). Manning Publications.

[8] Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. OSDI, 265-283.

[9] Otsu, N. (1979). A threshold selection method from gray-level histograms. IEEE Trans. SMC, 9(1), 62-66.

[10] Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. Journal of Big Data, 6, 60.

[11] Raza, A., et al. (2020). Automated blood group detection using image processing and neural networks. Biomedical Signal Processing and Control.

[12] Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. IEEE Trans. Knowledge and Data Engineering, 22(10), 1345-1359.

[13] Selvaraju, R. R., et al. (2017). Grad-CAM: Visual explanations from deep networks. ICCV, 618-626.

[14] Bradski, G., & Kaehler, A. (2008). Learning OpenCV. O'Reilly Media.

[15] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

[16] Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer.

[17] Chawla, N. V., et al. (2002). SMOTE: Synthetic minority over-sampling technique. JAIR, 16, 321-357.

[18] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. ICLR. arXiv:1412.6980.

[19] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training. ICML, 448-456.